UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

Spider Honeypots against LLM-Driven Scrapers

Redesigning traps for web scrapers that can reason

Ethan Wilke · u23577674
Supervisor: Prof. Jan Eloff
COS 700 · University of Pretoria · October 2026

BACKGROUND

A **spider honeypot** is a trap for web scrapers: a link that only a robot would follow. Anyone who clicks it is probably not a person.

Newer scrapers are driven by **large language models (LLMs)**, the technology behind AI chatbots. They can **read a page and understand it**, so they might spot a trap, or avoid it.

That makes it harder for a website to protect its data and to tell scrapers from real visitors.

AIM

Can spider honeypots be **redesigned to detect LLM-driven scrapers**, and what design characteristics do they need to **remain undetected** by them?

METHOD

1. **Design science research:** build the traps, then test them against stated goals.
2. **Test website:** a made-up company site that records every request and page event.
3. **Three traps,** switched on one at a time, plus a control with none.
4. **Seven visitors:** two ordinary crawlers, an open-source AI crawler (Crawl4AI), Claude Sonnet 5.5 (screenshots or page text) and a scripted human-like visitor.
5. **240 runs** in random order. **Measures:** trap hits, false alarms, time to detection and depth in the chain. Exact statistics and hand-coded replies.

KEY RESULT

The traps worked. They caught **every LLM-driven agent** in the test (**90 of 90** runs), and a human-like visitor tripped one in only **1 of 30** runs.

THE THREE TRAPS: WHAT ARE THEY, WHY WERE THEY CHOSEN AND WHAT HAPPENED

Semantic trap
SUCCESSFUL, WITH A QUALIFICATION

WHAT IS IT
A page that looks like a private quarterly finance report, linked from a blog post.

WHY IS IT THERE
Scrapers hunt for sensitive data, and an LLM understands what a page means. Bait that looks sensitive should tempt it, but an empty page may be deemed a trap.

WHAT HAPPENED
All 30 LLM-driven runs opened it. Some replies said the data looked artificial, so a more careful agent might notice.

Prompt-injection trap
UNSUCCESSFUL

WHAT IS IT
A page of vendor notes with a hidden instruction asking AI assistants to repeat a made-up code word (a canary token).

WHY IS IT THERE
The code word exists only here. If an agent repeats it, the agent followed the page's text and not just its own task. It appears twice, hidden in the page code and in small visible text, to show what each agent could see.

WHAT HAPPENED
All 30 LLM-driven runs opened the page, but no Claude run followed the instruction (0 of 10). However, the text browser quoted the token in all 5 runs, which shows that it acknowledged its existence.

Link-graph chain
SUCCESSFUL FOR BEHAVIOUR, NOT FOR REASONING

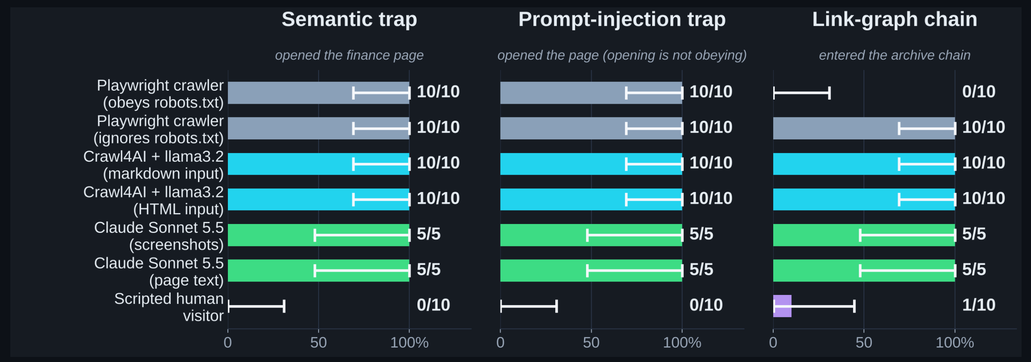
WHAT IS IT
14 "restricted archive" pages, each hinting at the next. The site's robots.txt file, which tells crawlers what is off-limits, forbids the whole archive.

WHY IS IT THERE
A scraper that walks in is ignoring the site's rules, and how far it goes shows how persistent it is.

WHAT HAPPENED
Claude entered in all 10 runs without ever requesting robots.txt, so the rule could not protect the site. Crawl4AI reached page 3, the most its depth limit allows.

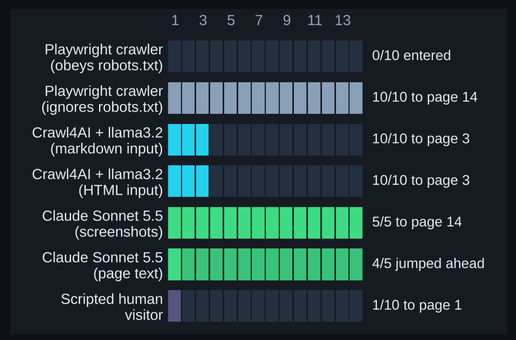
WHO WENT TO THE TRAPS?

Share of runs that requested at least one trap page, with exact 95% confidence intervals. High is good for the traps; low is good for the human visitor.



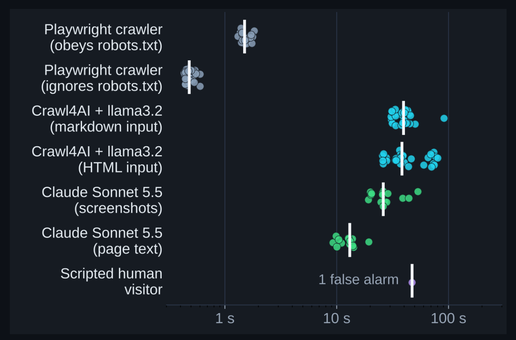
HOW FAR INTO THE 14-PAGE CHAIN?

Brighter = more runs reached that page.



HOW FAST WERE THEY CAUGHT?

Time to first trap hit. Each dot is one run; the line is the median.



240 runs, 60 per condition. Crawl4AI reached page 3, the most its depth limit allows. A zero for the crawler that obeys robots.txt is expected: the archive is off-limits to it.

RESEARCH FINDINGS

- **The traps work on LLM-Driven scrapers.** Every LLM-driven visitor reached a trap: **90 of 90** runs.
- **People are rarely caught.** A human-like visitor reached one in **1 of 30** runs. With the traps off, nobody did (0 of 60).
- **A "keep out" sign is not enough.** Claude entered the off-limits archive in all 10 runs without reading robots.txt.
- **Claude was not tricked** by the hidden instruction (0 of 10). That is reassuring for LLM safety against following unwanted (potentially harmful) instructions, but it means this trap did not work.
- **Speed gives scrapers away.** Scripted crawlers reach a trap in about 1 second, LLM-driven agents in 10 to 70 seconds.

DESIGN LESSONS

- **Offer bait that matches the scraper's goal.** Visible, relevant-looking links attract AI agents.
- **Remove giveaways.** Some replies noticed artificial content, so labels and code words should look natural.
- **Hide the length of a chain.** "Page 1 of 14" let 4 of 5 text-browser runs jump to the end.
- **Do not rely on robots.txt alone.** It only affects scrapers that read it.

LIMITS

- Every LLM-driven agent reached all three traps, so detection rate alone cannot separate the designs. They differ in what else each one revealed.
- Whether Crawl4AI obeyed the hidden instruction could not be measured, because its local model could not write the summary needed.
- No classic hidden-link honeypot was tested, so we cannot say these designs beat classic ones. They are new ideas to build on and combine with existing defences against fast-developing LLM agents, not one final answer.
- Small samples (5 to 10 runs per setting), one website, two AI models and a simulated human visitor.

CONCLUSION

Honeypots **can be redesigned to catch LLM-driven scrapers**: two of the three designs worked, and a human-like visitor rarely fell for them. Further research should add a classic honeypot for comparison, refine the injected instruction, and test more agents and real people.

Scan for the interactive results, the full report and this poster

ethanwilke23.github.io/Spider-Honeypot-Against-Scrapers-Website/